

Security Audit

1WalletWins (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	6
Security Rating	7
Intended Smart Contract Functions	8
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Conclusion	42
Our Methodology	43
Disclaimers	45
About Hashlock	46

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The 1WalletWins team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

1WalletWins is a DeFi raffle protocol built on EVM infrastructure that enables users to participate in on-chain, multi-tiered lottery-style draws using stablecoins (e.g., USDC). The system leverages Chainlink VRF for verifiable randomness and Chainlink Automation for lifecycle management, ensuring transparent and trustless winner selection. Its architecture includes configurable raffle tiers with time-based state transitions (OPEN → EARLY_DRAW → FINALISED → SETTLED), an exit and inactivity handling mechanism, and a jackpot system with probabilistic triggers. The protocol emphasizes fairness, automation, and reliability through deterministic state management, queue-based settlement, and modular smart contracts, creating a decentralized and transparent raffle ecosystem.

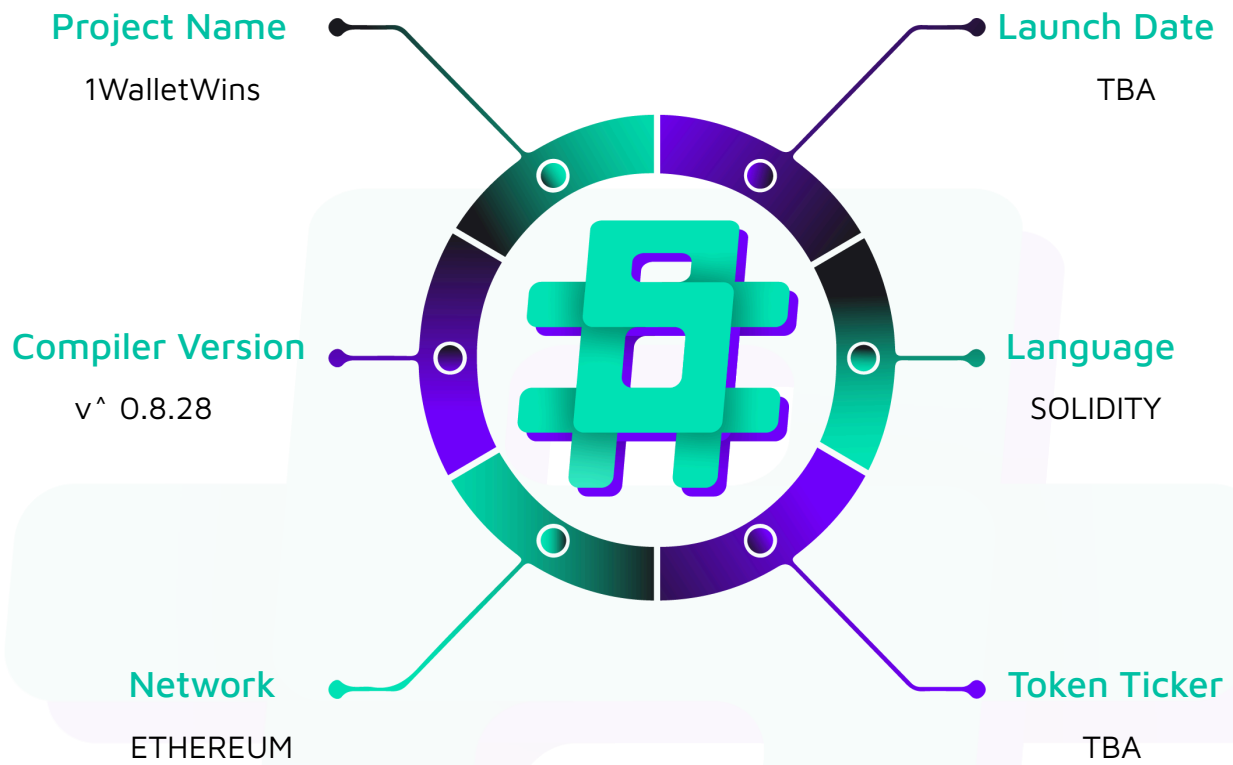
Project Name: 1WalletWins

Project Type: DeFi

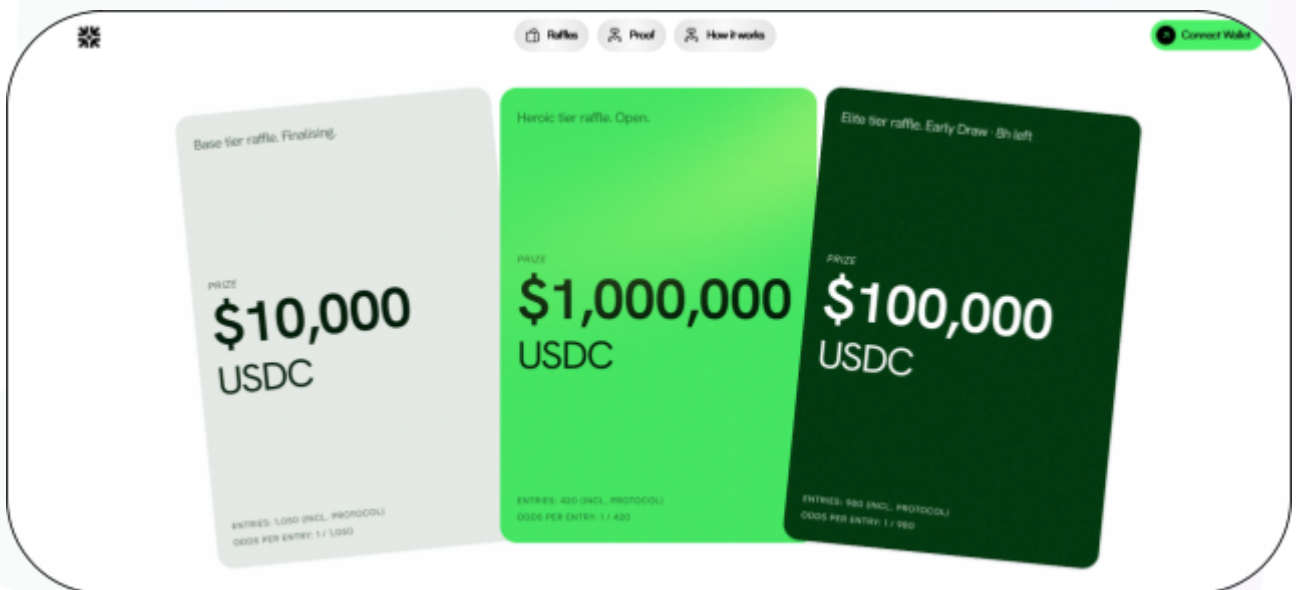
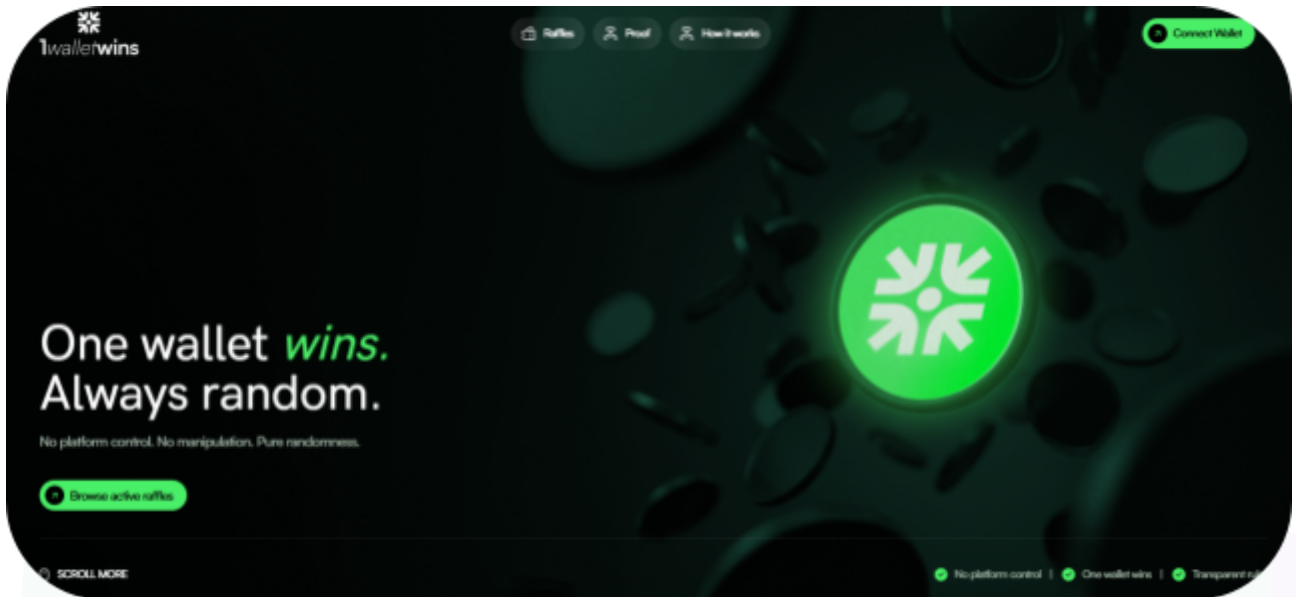
Compiler Version: ^ 0.8.28

Website: 1walletwins.com



Visualised Context:

Project Visuals



Audit Scope

We at Hashlock audited the solidity code within the 1WalletWins project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	1WalletWins Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	April, 2026
Contract 1	packages/contracts/contracts/RaffleManager.sol
Contract 2	packages/contracts/contracts/Jackpot.sol
Contract 3	packages/contracts/contracts/libraries/BinarySearch.sol
Contract 4	packages/contracts/contracts/proxy/ERC1967ProxyHelper.sol
Contract 5	packages/contracts/contracts/interfaces/IRaffleManager.sol
Contract 6	packages/contracts/contracts/interfaces/IJackpot.sol
Audited GitHub Commit Hash	3017cd6e44a94ac672de5c093c9d8d80f2591e84
Fix Review GitHub Commit Hash	17793a2bdcc3e20641780b838d85daa3c95a0886

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 High severity vulnerabilities

2 Medium severity vulnerabilities

13 Low severity vulnerabilities

4 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
RaffleManager.sol - Allows the admin (ADMIN_ROLE) to: - Create raffle tiers - Configure existing raffle tiers - Manually create new raffles for a tier - Set VRF configuration (subscription, key hash, gas limit, confirmations) - Recover accidentally sent tokens (surplus only for USDC) - Allows the default admin (DEFAULT_ADMIN_ROLE) to: - Authorize UUPS contract upgrades - Grant and revoke roles - Allows any caller to: - Trigger raffle upkeep (settle winners, transition states, request VRF) - Allows users to: - Buy raffle tickets - Claim unclaimed prize winnings - Allows Chainlink VRF Coordinator to: - Fulfill random word requests	Contract achieves this functionality.
Jackpot.sol - Allows the admin (ADMIN_ROLE) to: - Set jackpot trigger chance (BPS) - Set jackpot draw delay - Allows any caller to: - View the current jackpot pool	Contract achieves this functionality.

- | | |
|---|--|
| <ul style="list-style-type: none">- View the raffle IDs contributing to the current jackpot | |
|---|--|



Code Quality

This audit scope involves the smart contracts of the 1WalletWins project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the 1WalletWins project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] Jackpot#_processJackpotUpkeep - Jackpot funds permanently lost when protocol address wins the jackpot

Description

When the jackpot settles, a two-stage winner selection is performed: first a winning raffle is selected (weighted by USDC contribution), then a random ticket within that raffle is selected via `_getRaffleWinnerByTicket`. Because protocol tickets are included in the raffle's `entries` array, the contract address (`address(this)`) can be selected as the jackpot winner.

Vulnerability Details

During raffle settlement in `_settleRaffles` (L498-503), protocol tickets are appended to the raffle's `entries` array with `entryOwners` set to `address(this)`:

```
uint128 startId = uint128(r.ticketCount);
r.entries.push(BinarySearch.Entry({start: startId, size: uint128(protocolTickets)}));
entryOwners[_packEntryKey(raffleId, r.entries.length - 1)] = address(this);
```

When `_processJackpotUpkeep` (Jackpot.sol L140-178) later settles the jackpot, `_getRaffleWinnerByTicket` can return `address(this)` if the randomly selected ticket falls within the protocol's entry range. The subsequent `_transferPrize(address(this), pool)` call executes a USDC self-transfer (no-op on balance), but `totalAccountedUsdc` is decremented by the full pool amount:

```
// RaffleManager.sol L708-721
function _transferPrize(address winner, uint128 amount) internal override returns
(bool) {
    try usdc.transfer(winner, amount) returns (bool success) {
        if (success) {
```

```

        totalAccountedUsdc -= amount; // Decrement despite self-transfer
        return true;
    }
    // ...
}
}

```

After `_resetJackpotState()` clears all jackpot entries, the entire pool becomes unaccounted surplus recoverable by the admin via `recoverTokens`.

Impact

The entire jackpot pool (potentially accumulated over many raffles) is permanently lost to participants. The probability scales with `protocolTicketPercentageBPS` at 5% (500 BPS); the expected probability per jackpot settlement is approximately 4.76% ($\text{protocolTickets} / \text{totalTickets} = 5/105$). The admin can then drain the surplus via `recoverTokens`, effectively extracting funds that belong to jackpot participants.

Recommendation

We recommend modifying the `_processJackpotUpkeep` function to check whether the winner is the contract itself and, if so, carry the pool forward to the next jackpot round instead of resetting the state:

```

address winner = _getRaffleWinnerByTicket(winningRaffleId, entrySelector);

if (winner == address(this)) {
    jackpotTriggeredAt = 0;
    jackpotVrfPending = false;
    return;
}

```

Status

Resolved

[H-02] RaffleManager#_getAdjustedRaffleState - Raffle funds permanently locked when minTickets threshold is never reached

Description

The `_getAdjustedRaffleState` function requires `ticketCount >= minTickets` before any state progression from `OPEN` can occur. If a raffle sells some tickets but never reaches `minTickets`, it remains in `OPEN` state indefinitely with no mechanism to cancel, refund, or force-settle.

Vulnerability Details

In `_getAdjustedRaffleState` (L602-638), the only paths out of `OPEN` state are:

1. **Sold out:** `ticketCount == maxTickets` (L617)
2. **Inactivity with min tickets met:** `ticketCount >= minTickets` (L622)

```
// L622-630 - Both transitions require minTickets
if (rawState == Lifecycle.OPEN && ticketCount >= minTickets) {
    if (block.timestamp > createdAt + earlyDrawThresholdSeconds) {
        if (block.timestamp > lastPurchasedAt + inactivityThresholdSeconds) {
            return Lifecycle.FINALISED;
        }
        return Lifecycle.EARLY_DRAW;
    }
}
```

When `ticketCount > 0` but `ticketCount < minTickets`, no amount of elapsed time triggers a state change. The contract has no `cancelRaffle` or `refund` function. The `createRaffle` admin function (L266-273) requires the current raffle to be `SETTLED`, so the entire tier is bricked.

User funds are tracked in `totalAccountedUsdc`, preventing recovery via `recoverTokens`.

Impact

User USDC is permanently locked in the contract. The affected tier cannot create new raffles. The only recovery path is a full UUPS contract upgrade, which affects all contract state and requires significant coordination.



Recommendation

We recommend adding a timeout-based finalization path in `_getAdjustedRaffleState` for raffles below `minTickets`:

```
// After the existing minTickets check, add:
if (rawState == Lifecycle.OPEN && ticketCount > 0 && ticketCount < minTickets) {
    if (block.timestamp > lastPurchasedAt + 2 * inactivityThresholdSeconds) {
        return Lifecycle.FINALISED;
    }
}
```

Alternatively, we recommend adding an admin `cancelRaffle` function that refunds all ticket buyers and resets the tier.

Status

Resolved

[H-03] RaffleManager#_settleRaffles - Protocol raffle win sends 100% to jackpot instead of intended 20%

Description

When the protocol wins a raffle, `_settleRaffles` passes the entire raffle balance to `_handleJackpotWin` as the jackpot contribution. The client has confirmed the intended split is 20% to jackpot and 80% as protocol revenue, but no split logic exists in the code.

Vulnerability Details

At `RaffleManager.sol` L525-527, the full `prize` (equal to `r.balance`) is passed to `_handleJackpotWin`:

```
if (winner == address(this)) {  
    _handleJackpotWin(raffleId, randomWord, prize); // prize = r.balance (100%)  
}
```

`_handleJackpotWin` (`Jackpot.sol` L104) then uses the full amount as both the jackpot entry weight and the USDC reserved for the pool:

```
jackpotEntries.push(BinarySearch.Entry({start: startWeight, size: _jackpotAmount}));
```

There is no mechanism to split the protocol win into a jackpot portion and a protocol revenue portion. The entire raffle balance is locked for the jackpot, and no USDC is transferred to a protocol revenue address.

Impact

The protocol receives 0% revenue from raffle wins instead of the intended 80%. All protocol raffle win funds are allocated to the jackpot pool, inflating it beyond the intended design. This directly affects protocol economics; the protocol ticket mechanism was designed to generate revenue, but all revenue is redirected to the jackpot.

Recommendation

We recommend modifying `_settleRaffles` to split the protocol win between jackpot (20%) and protocol revenue (80%), with the split ratio configurable:

```
if (winner == address(this)) {
```

```
uint128 jackpotShare = prize * jackpotShareBPS / BPS_DENOMINATOR;  
uint128 protocolRevenue = prize - jackpotShare;  
  
if (jackpotShare > 0) {  
    _handleJackpotWin(raffleId, randomWord, jackpotShare);  
}  
if (protocolRevenue > 0) {  
    _transferPrize(protocolRevenueAddress, protocolRevenue);  
}  
}
```

A configurable `jackpotShareBPS` (e.g., 2000 = 20%) and a `protocolRevenueAddress` should be added as state variables with admin setters.

Status

Resolved

Medium

[M-01] Jackpot#setJackpotDrawDelay - Admin can retroactively alter pending jackpot timing

Description

The `setJackpotDrawDelay` function can be called during an active jackpot countdown, allowing the admin to extend or shorten the delay after the jackpot has already been triggered.

Vulnerability Details

At `Jackpot.sol` L71-78, there is no guard preventing modification while `jackpotTriggeredAt > 0`:

```
function setJackpotDrawDelay(uint32 _jackpotDrawDelay) external override
onlyRole(ADMIN_ROLE) {
    if (_jackpotDrawDelay == 0) revert InvalidJackpotDrawDelay();

    uint32 oldValue = jackpotDrawDelay;
    jackpotDrawDelay = _jackpotDrawDelay;

    emit JackpotDrawDelayUpdated(oldValue, _jackpotDrawDelay);
}
```

The `_checkJackpotUpkeep` function (L129-133) uses the current `jackpotDrawDelay` value:

```
function _checkJackpotUpkeep() internal view returns (bool needsUpkeep) {
    needsUpkeep = jackpotTriggeredAt > 0
        && !jackpotVrfPending
        && block.timestamp >= jackpotTriggeredAt + jackpotDrawDelay;
}
```

An admin can set `jackpotDrawDelay` to an extremely large value (e.g., 365 days) after the jackpot has triggered, effectively freezing the jackpot indefinitely. Conversely, reducing the delay rushes settlement before high-value raffles contribute entries.

Impact

Admin can manipulate jackpot timing after the trigger event, affecting fairness for all participants. Extending the delay locks jackpot funds and delays payouts. Reducing it can settle the jackpot before expected entries accumulate, benefiting existing entry holders.

Recommendation

We recommend adding a guard in the `setJackpotDrawDelay` function to prevent changes while a jackpot is triggered:

```
function setJackpotDrawDelay(uint32 _jackpotDrawDelay) external override
onlyRole(ADMIN_ROLE) {
    if (_jackpotDrawDelay == 0) revert InvalidJackpotDrawDelay();
    if (jackpotTriggeredAt > 0) revert JackpotAlreadyTriggered();

    uint32 oldValue = jackpotDrawDelay;
    jackpotDrawDelay = _jackpotDrawDelay;

    emit JackpotDrawDelayUpdated(oldValue, _jackpotDrawDelay);
}
```

Status

Resolved

[M-02] RaffleManager#constructor - Missing `_disableInitializers()` allows attacker to claim the implementation contract

Description

The `RaffleManager` contract has no constructor calling `_disableInitializers()`. After deployment, the implementation contract remains uninitialized and any attacker can call `initialize()` directly on it, gaining `DEFAULT_ADMIN_ROLE` and `ADMIN_ROLE`.

Vulnerability Details

The `RaffleManager` contract has no constructor:

```
contract RaffleManager is
    VRFConsumerBaseV2Upgradeable,
    UUPSUpgradeable,
    Jackpot,
    ReentrancyGuardUpgradeable,
    IRaffleManager
{
    // No constructor - _disableInitializers() is never called

    function initialize(...) public initializer {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(ADMIN_ROLE, msg.sender);
    }
}
```

OpenZeppelin's [UUPS upgradeable) explicitly recommends:

> _To avoid leaving the implementation contract uninitialized, you should invoke the `_disableInitializers` function in the constructor._

The Ignition deployment flow:

1. `m.contract("RaffleManager")`, deploys implementation at `IMPL`. No constructor runs.

****Implementation is UNINITIALIZED.****

2. `m.contract("ERC1967ProxyHelper", [IMPL, initData])`, deploys proxy, calls `initialize` via `delegatecall`. ****Proxy is initialized. Implementation is still UNINITIALIZED.****

After deployment, an attacker calls `initialize()` on `IMPL` directly, becoming the admin on the implementation contract.

The severity is mitigated by OZ 4.9.6's `onlyProxy` modifier on `upgradeTo` and `upgradeToAndCall`, which prevents the attacker from upgrading or destroying the implementation. The proxy's storage and functionality are unaffected. On pre-Cancun EVM chains (where `selfdestruct` is not restricted by EIP-6780), this would be ****Critical****, the attacker could upgrade the implementation to a contract that self-destructs, permanently bricking the proxy. Base Mainnet (target chain) supports Cancun, so this vector is currently blocked.

Impact

- Attacker gains `DEFAULT_ADMIN_ROLE` and `ADMIN_ROLE` on the implementation contract.
- Attacker can execute any admin function on the implementation directly (`createTier`, `setVrfConfig`, `configureTier`, etc.), affecting the implementation's own storage.
- The proxy is NOT directly affected (OZ's `onlyProxy` blocks `upgradeTo/upgradeToAndCall`).
- Violates OpenZeppelin's explicit security guidance, flagged by all major audit tooling.

Recommendation

We recommend adding a constructor to the `RaffleManager` contract that disables initializers:

```
/// @custom:oz-upgrades-unsafe-allow constructor
constructor() {
    _disableInitializers();
}
```

Status

Resolved

Low

[L-01] Jackpot#__Jackpot_init - Missing validation for jackpotTriggerChanceBPS during initialization

Description

The `__Jackpot_init` initializer sets `jackpotTriggerChanceBPS` without bounds validation, while the runtime setter `_setJackpotTriggerChanceBPS` correctly enforces `> 0` and `<= BPS_DENOMINATOR`.

Vulnerability Details

The initializer at `Jackpot.sol` L55-59 stores the value directly:

```
function __Jackpot_init(uint16 _jackpotTriggerChanceBPS, uint32 _jackpotDrawDelay)
internal onlyInitializing {
    jackpotTriggerChanceBPS = _jackpotTriggerChanceBPS; // No validation
    if (_jackpotDrawDelay == 0) revert InvalidJackpotDrawDelay();
    jackpotDrawDelay = _jackpotDrawDelay;
}
```

Compared to the setter at L199-201:

```
function _setJackpotTriggerChanceBPS(uint16 _jackpotTriggerChanceBPS) internal {
    if (_jackpotTriggerChanceBPS == 0 || _jackpotTriggerChanceBPS > BPS_DENOMINATOR)
    {
        revert InvalidJackpotTriggerChance();
    }
    // ...
}
```

Initializing with `0` means the jackpot can never trigger, all protocol raffle win funds accumulate indefinitely with no payout mechanism. Initializing with `> 10000` means the jackpot triggers on every protocol win.

Impact

If deployed with an invalid trigger chance, the jackpot mechanism is permanently broken from deployment. A value of `0` locks all protocol raffle win funds in the jackpot

pool with no way to trigger a draw. This is only correctable via the admin setter post-deployment, but the window between deployment and correction could accumulate significant locked funds.

Recommendation

We recommend adding the same validation in the `__Jackpot_init` function as exists in `_setJackpotTriggerChanceBPS`:

```
function __Jackpot_init(uint16 _jackpotTriggerChanceBPS, uint32 _jackpotDrawDelay)
internal onlyInitializing {
    if (_jackpotTriggerChanceBPS == 0 || _jackpotTriggerChanceBPS > BPS_DENOMINATOR)
    {
        revert InvalidJackpotTriggerChance();
    }
    jackpotTriggerChanceBPS = _jackpotTriggerChanceBPS;
    if (_jackpotDrawDelay == 0) revert InvalidJackpotDrawDelay();
    jackpotDrawDelay = _jackpotDrawDelay;
}
```

Status

Resolved

[L-02] RaffleManager#_calculateProtocolTickets - Protocol ticket rounding to zero for small raffles

Description

`_calculateProtocolTickets` uses integer division, causing protocol tickets to round to zero for small `ticketCount` values.

Vulnerability Details

```
function _calculateProtocolTickets(uint32 maxTickets, uint32
protocolTicketPercentageBPS) internal pure returns (uint32) {
    return uint32(uint256(maxTickets) * protocolTicketPercentageBPS /
BPS_DENOMINATOR);
}
```

For `ticketCount = 10` and `protocolTicketPercentageBPS = 500` (5%): $10 * 500 / 10000 = 0$. The protocol receives no tickets, preventing that raffle from contributing to the jackpot pool.

Impact

Low-tier raffles with few tickets never contribute to the jackpot, creating an imbalance where only high-ticket-count raffles feed the pool.

Recommendation

We recommend documenting this behavior as intentional, or using ceiling division when the product is non-zero.

Status

Acknowledged

[L-03] RaffleManager#_validateConfig - No upper bound validation on protocolTicketPercentageBPS

Description

_validateConfig allows protocolTicketPercentageBPS values up to 9999 (99.99%).

Vulnerability Details

```
if (config.protocolTicketPercentageBPS >= BPS_DENOMINATOR) revert  
InvalidProtocolTicketPercentage();
```

This only rejects values ≥ 10000 . Combined with H-01, extreme values dramatically increase the probability of jackpot funds being lost via protocol self-transfer.

Impact

An admin could configure a tier where the protocol receives up to 99.99% of tickets at settlement, making user wins extremely unlikely and amplifying the H-01 jackpot loss vector.

Recommendation

We recommend introducing a reasonable cap in the _validateConfig function (e.g., 2000 BPS = 20%).

Status

Acknowledged

[L-04] RaffleManager#_transferPrize - USDC-blacklisted winners cannot claim prizes

Description

When a settlement's `_transferPrize` fails for a blacklisted winner, the amount is stored in `unclaimedPrizes`. However, `claimPrize` uses `safeTransfer`, which also reverts for a blacklisted address.

Vulnerability Details

```
// _transferPrize - fails, stores in unclaimedPrizes
catch {
    unclaimedPrizes[winner] += amount;
    return false;
}

// claimPrize - also fails for blacklisted address
function claimPrize() external override nonReentrant {
    uint128 amount = unclaimedPrizes[msg.sender];
    // ...
    usdc.safeTransfer(msg.sender, amount); // Reverts for blacklisted
}
```

No admin override exists to redirect these funds to an alternative address.

Impact

Prize funds for USDC-blacklisted addresses are permanently locked in `unclaimedPrizes` with no recovery mechanism.

Recommendation

We recommend adding an admin function to redirect unclaimed prizes to an alternative address after a configurable timeout.

Status

Acknowledged

[L-05] RaffleManager#performUpkeep - VRF request revert rolls back pending settlements

Description

If VRF configuration is unset (defaults to zero), the VRF request in Step 3 of `performUpkeep` reverts at the coordinator, rolling back all state changes from Step 1 (VRF settlements) in the same transaction.

Vulnerability Details

Steps 1-3 execute sequentially in a single transaction. A revert in Step 3 (`vrfCoordinator.requestRandomWords`) undoes all settlements from Step 1. The `maxVrfSettlements` workaround exists but is not part of the standard Chainlink Automation path (empty `performData`).

Impact

Pending VRF settlements and raffle state transitions are blocked if VRF config is invalid.

Recommendation

We recommend guarding VRF requests with a config validity check before Step 3.

Status

Acknowledged

[L-06] RaffleManager#_settleRaffles - VRF random word reused for winner selection and jackpot trigger

Description

The same VRF `randomWord` is used to select the raffle winner (`randomWord % ticketCount`) and to determine the jackpot trigger (`randomWord % 10000 < triggerChanceBPS`), creating a weak statistical correlation.

Vulnerability Details

```
// _settleRaffles L507
uint128 winningTicket = uint128(randomWord % r.ticketCount);

// _handleJackpotWin L119 (same randomWord)
bool drawJackpot = _randomWords % BPS_DENOMINATOR < jackpotTriggerChanceBPS;
```

Impact

The correlation is statistically weak and not exploitable (VRF values are revealed post-commitment), but deviates from best practices for using randomness.

Recommendation

We recommend deriving an independent value for the jackpot trigger check via `keccak256(abi.encodePacked(randomWord, "jackpot_trigger"))`.

Status

Acknowledged

[L-07] RaffleManager#_getAdjustedRaffleState - earlyDrawThresholdSeconds documentation mismatch

Description

The README and `vrf-upkeep-flow.md` describe the OPEN → FINALISED transition as "2x the inactivity threshold." The code uses two independent thresholds measured from different reference points and requires `ticketCount >= minTickets`, which the documentation omits.

Vulnerability Details

```
// OPEN → EARLY_DRAW: time since CREATION, not inactivity
block.timestamp > createdAt + earlyDrawThresholdSeconds

// EARLY_DRAW → FINALISED: time since LAST PURCHASE
block.timestamp > lastPurchasedAt + inactivityThresholdSeconds
```

Impact

Operators configuring the system based on documentation may set unexpected threshold combinations.

Recommendation

We recommend updating the README and `vrf-upkeep-flow.md` to accurately describe the dual-threshold system, reference timestamps, and `minTickets` prerequisite.

Status

Resolved

[L-08] README - Production deployment example missing _jackpotDrawDelay parameter

Description

The `initialize` function requires 4 parameters but the README production module example only passes 3, `_jackpotDrawDelay` is missing.

Vulnerability Details

```
// README example - missing 4th parameter
const initData = m.encodeFunctionCall(implementation, "initialize", [
  usdc, vrfCoordinator, jackpotTriggerChanceBPS,
]);

// Actual function signature
function initialize(IERC20 _usdc, address _vrfCoordinator, uint16
_jackpotTriggerChanceBPS, uint32 _jackpotDrawDelay)
```

Impact

Following the README example directly causes deployment failure.

Recommendation

We recommend updating the README example to include all 4 parameters and adding `jackpotDrawDelay` to the parameters JSON example.

Status

Resolved

[L-09] raffle-manager-production#module - Tiers created atomically before VRF configuration

Description

The production Ignition module creates all four raffle tiers in the same deployment batch but does not configure VRF. This creates a window where raffles are open for ticket purchases but `performUpkeep` will revert if a raffle needs finalization.

Vulnerability Details

In `raffle-manager-production.ts` (L52-62), all four tiers are created immediately after proxy deployment with no `setVrfConfig` call. Compare with the testnet module (`raffle-manager-testnet.ts` L85-93), which atomically sets VRF config after tier creation.

The production deployment timeline:

1. Deploy → 4 tiers created → 4 raffles immediately OPEN.
2. Users can buy tickets via `buyTicket()`.
3. If a raffle sells out or times out → `performUpkeep` Step 2 transitions to FINALISED.
4. Step 3 calls `vrfCoordinator.requestRandomWords(...)` with zero config → `**reverts**`.
5. The revert rolls back Step 2's state transitions as well.
6. Admin manually calls `setVrfConfig()` → upkeep resumes.

Impact

Between deployment and VRF configuration, any raffle reaching finalization causes `performUpkeep` to revert entirely, temporarily blocking all upkeep. Users who bought tickets are not at risk of fund loss, but the raffle is stuck until VRF is configured.

Recommendation

We recommend including `setVrfConfig` in the production Ignition module before tier creation, or reversing the order so raffles are never open without a working VRF configuration.

Status

Resolved

[L-10] RaffleManager#fulfillRandomWords - No validation that requestId is known

Description

The VRF callback stores random words and enqueues the request without verifying the `requestId` was previously recorded. An unknown `requestId` defaults to `VrfRequestType.RAFFLE` (enum value 0). Only the VRF coordinator can call this, so exploitation requires a compromised coordinator.

Recommendation

We recommend adding a check that the request was previously initialized.

Status

Acknowledged

[L-11] RaffleManager#buyTicket - Unsafe downcast from uint256 to uint128

Description

`r.balance += uint128(cost)` and `totalAccountedUsdc += uint128(cost)` silently truncate if `cost > type(uint128).max`. Practically unexploitable because `safeTransferFrom` would fail first, no account holds enough USDC for the truncation to matter.

Recommendation

We recommend adding `require(cost <= type(uint128).max)` for defense-in-depth.

Status

Acknowledged

[L-12] RaffleManager#buyTicket - Allows purchasing on any historic raffle

Description

`buyTicket` accepts any `raffleId` without checking it matches `currentRaffleId[tierId]`. In normal operation this is safe (settled raffles reject via state check), but combined with H-02, users could buy tickets for a stuck raffle that will never settle.

Recommendation

We recommend adding a check that the raffle is the current active raffle for its tier.

Status

Resolved

[L-13] raffle-manager-production#module - No role transfer to multisig after deployment

Description

The production deployment grants `DEFAULT_ADMIN_ROLE` and `ADMIN_ROLE` to `msg.sender` (the deployer EOA) during `initialize()`. The module does not include a step to transfer `DEFAULT_ADMIN_ROLE` to a multisig or governance contract.

Vulnerability Details

```
_grantRole(DEFAULT_ADMIN_ROLE, msg.sender);  
_grantRole(ADMIN_ROLE, msg.sender);
```

`DEFAULT_ADMIN_ROLE` controls `_authorizeUpgrade` (L655), meaning the deployer EOA can unilaterally upgrade the contract to arbitrary code. If the deployer key is compromised, the attacker gains full control.

Recommendation

We recommend adding role transfer steps to the production deployment procedure: grant `DEFAULT_ADMIN_ROLE` to a multisig, then revoke it from the deployer EOA. Consider including these steps in the Ignition module or documenting them as mandatory post-deploy actions.

Status

Resolved

QA

[Q-01] RaffleManager#_transferPrize - Inconsistent use of raw transfer vs SafeERC20

Description

`_transferPrize` uses raw `usdc.transfer` wrapped in try/catch, while `buyTicket` uses `safeTransferFrom` and `claimPrize` uses `safeTransfer`. The try/catch approach is intentionally chosen to avoid reverting during batch settlement but is undocumented.

Recommendation

We recommend adding inline documentation explaining the design decision.

Status

Resolved

[Q-02] RaffleManager#configureTier - Tier version counter saturates at uint16 maximum

Description

`tierVersion[tierId]` is `uint16` (max 65535). After 65535 `configureTier` calls per tier, the increment reverts due to Solidity 0.8.x overflow checks, permanently preventing further config changes.

Recommendation

We recommend documenting this limit or using `uint32` for the version counter.

Status

Resolved

[Q-03] RaffleManager#_calculateProtocolTickets - Misleading parameter name

Description

The function parameter is named `maxTickets` but is called with `r.ticketCount` at L496. The misleading name could cause confusion during maintenance.

Recommendation

We recommend renaming the parameter to `ticketsSold`.

Status

Resolved

[Q-04] RaffleManager#_settleRaffles - VRF word assignment mismatch across multi-round upkeep

Description

The README claims "the tier that requested first will get the first fulfillment's random word." `_settleRaffles` assigns words by ascending tier ID, not by request order. When multiple `performUpkeep` rounds interleave before VRF fulfillment, a raffle finalized in a later round with a lower tier ID consumes words from an earlier VRF response. Not exploitable since all VRF words are independently random.

Recommendation

We recommend updating the README to document that words are assigned by tier order, not request order.

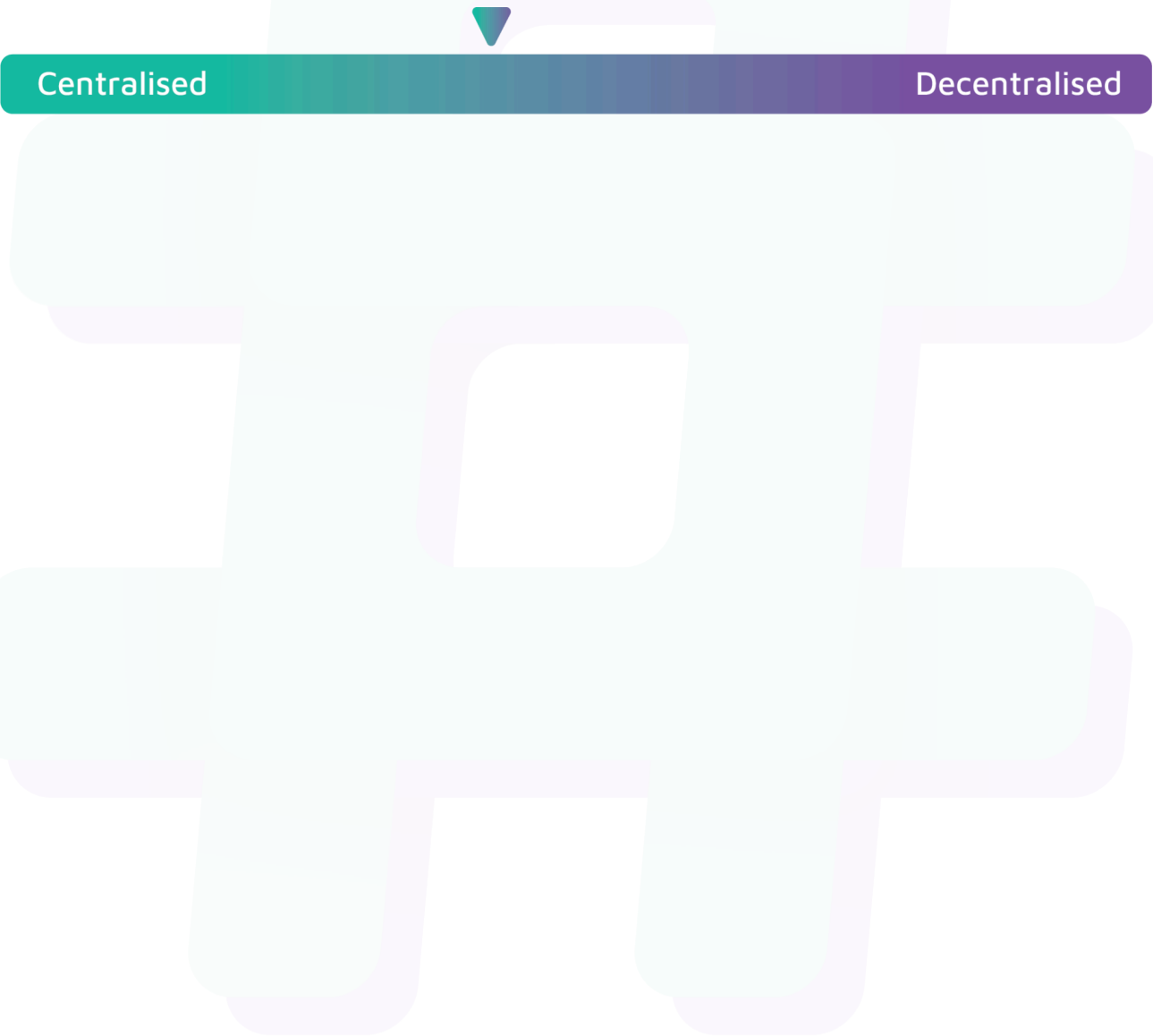
Status

Resolved

Centralisation

The 1WalletWins project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the 1WalletWins project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.